GRADUATION PROJECT · 2026

VisionLink

A Wearable Industrial Assistant on Raspberry Pi 4B

Master Technical Reference

Team: Alaa · Ali Salih · F-Alaa · Defne

Branch `openai-sdk` · 2026-05-09

VisionLink — Master Reference

A wearable industrial assistant on Raspberry Pi 4B

Last updated: 2026-05-09 · Branch: `openai-sdk` · Team: Alaa · Ali Salih · F-Alaa · Defne

This is the single source of truth. Everything else in `Documentation/` is per-session work logs. If something here disagrees with another file, **this file wins**.

---

1. What VisionLink Is

VisionLink is a wearable assistant for factory workers, built like smart glasses (or a helmet attachment) and powered by a Raspberry Pi 4B. The worker has six physical buttons within reach. Each button triggers a different hands-free workflow — talking to AI, raising an emergency, or documenting their shift with photos, videos, and voice notes.

Everything the worker captures is sent to a cloud database where a supervisor watches it stream in live on a web dashboard.

The six buttons split into two groups:

Group	Buttons	Purpose
AI Assistant	Agent · Agent + Vision · SOS	Talk to a cloud AI that can see, search the parts catalog, log incidents, request spare parts, send emails, or trigger an emergency
Documentation	Session · Voice · Visual	Hands-free recording of work activity — sessions, photos, videos, voice notes

2. The Six Buttons (canonical naming)

#	Button	What it does	GPIO	Pi Pin	Status
1	**Agent**	Audio-only AI conversation. Single press starts, double press stops.	5	Pin 29	✅ working
2	**Agent + Vision**	AI session that sees through the camera. Single = start (snaps a frame), double = stop.	6	Pin 31	✅ working
3	**SOS**	Panic mode. Single press = warning toast (pocket-dial protection). **Double press** = full SOS — emails safety officer, starts AI session, streams live frames.	13	Pin 33	✅ working
4	**Documentation Session**	Toggle a doc session — single press opens, single press again closes. 800 ms cooldown absorbs button bounce.	23	Pin 16	✅ working
5	**Voice**	Press-to-toggle voice note — single press starts recording, single press stops + uploads. 800 ms cooldown.	22	Pin 15	✅ working
6	**Visual**	Single = photo, Double = 5-second video (no audio).	25	Pin 22	✅ wired, **bounce issue — see Open Issues §11**

> **Code-name footnote.** The Python code internally uses an older numbering (`BTN_SESSION` , `BTN_PHOTO_VIDEO` , `BTN_VOICE_NOTE` , `BTN_AI_CAMERA` , `BTN_AI_VOICE` , `BTN_AI_AGENT` in `config.py` and `b1_…` through `b6_…` in `src/subsystems/button_handlers.py`). The mapping above is correct — only the names differ. We keep the internal names as-is to avoid touching working code.

3. The Hardware

VisionLink runs on consumer Raspberry Pi hardware with off-the-shelf sensors. Total parts cost is under €120.

3.1 Components

Component	Model	Role	Connection
Computer	Raspberry Pi 4B (8 GB RAM)	The brain. Runs all software locally.	USB-C 5 V / 3 A
Camera	Pi Camera Module 3	Photos, videos, live frames for AI vision and SOS	CSI ribbon cable
Microphone	SPH0645LM4H (GY-SPH0645 board)	Voice input — for the AI and voice notes	I²S digital — GPIO 18 (BCLK), 19 (LRCLK), 20 (data)
Amplifier	MAX98357A	Drives the speaker from a digital I²S signal	I²S digital — GPIO 18 (BCLK), 19 (LRCLK), 21 (data)
Speaker	8 Ω 1 W oval (Adafruit-style)	Audio output	Screw terminals on the amp
Buttons	6 × tactile push-button switches	Worker input	GPIO 5, 6, 13, 22, 23, 25 with shared GND
Power	USB-C power supply or USB power bank	Standalone power for the wearable	USB-C

3.2 Complete Pin Map (current, verified)

The Pi 4B has a 40-pin GPIO header. VisionLink uses these pins:

Pin	GPIO	What it carries
1	—	**3.3 V power** → Mic VDD
2	—	**5 V power** → Amp VIN
6	—	GND → Mic GND
9	—	GND → Amp GND
12	18	I²S BCLK — shared between mic and amp
14	—	GND → spare for buttons
15	22	**Voice button** signal
16	23	**Session button** signal
20	—	GND → button common
22	25	**Visual button** signal
29	5	**Agent button** signal
31	6	**Agent + Vision button** signal
33	13	**SOS button** signal
35	19	I²S LRCLK — shared between mic and amp
38	20	Mic DOUT — audio bits flowing into the Pi
39	—	GND → Mic SEL (often tied here)
40	21	Amp DIN — audio bits flowing out of the Pi

3.3 Wiring Philosophy

Each button has **two wires**: signal goes to the GPIO pin, and the diagonally-opposite leg goes to any GND. Pressing the button shorts the pin to ground; the Pi's internal pull-up resistor detects the falling edge.

The microphone and amplifier share I²S clock lines (BCLK on GPIO 18, LRCLK on GPIO 19) but have separate data lines (mic DOUT = GPIO 20, amp DIN = GPIO 21). All audio runs on four wires plus power and ground.

---

4. The Six Buttons — Detailed Behavior

4.1 Agent button (GPIO 5, Pin 29)

Audio-only AI conversation. The cloud AI hears you and speaks back through the I²S amplifier and speaker — no camera involved.

Single press: Open a streaming WebSocket to either Google Gemini Live or OpenAI Realtime (the supervisor picks the provider in the *Wearable Settings* panel). Audio flows in real time both ways. The AI has access to all six tools (parts catalog, incident logging, etc.) and can act on the worker's behalf.

Double press: Close the AI session.

4.2 Agent + Vision button (GPIO 6, Pin 31)

Same as Agent, plus the AI sees through the Pi Camera.

Single press: Start an AI session AND inject a fresh camera snapshot so the AI knows what the worker is looking at. Subsequent single presses while a session is live inject another fresh frame.

Double press: Close the AI session.

Vision modes (set in Wearable Settings):

- `snap_on_press` — default; one image per press
- `gemini_video` — continuous 1 fps stream (Gemini-only)
- `auto_snap_4s` — auto-snap every 4 s

4.3 SOS button (GPIO 13, Pin 33) — emergency

Single press: A toast warns *"double-click required for SOS"*. Pocket-dial protection. No real action.

Double press (within 500 ms): Full SOS panic mode:

1. Insert a row in the `sos_events` table.
2. Auto-close any open documentation session (worker shouldn't be doing paperwork in an emergency).
3. Email the safety officer via Gmail SMTP.
4. Start an OpenAI Realtime session with a calm-emergency prompt (*"Stay calm, ask short questions, describe what you see, tell them help is on the way"*). OpenAI is used regardless of normal Agent provider settings.
5. Auto-snap a fresh camera frame every 10 s; upload each to storage; show in the supervisor's SOS panel. The supervisor can flip a *resolved* flag to remotely shut everything off; the wearable polls every 2 s and tears down within ~2 s of the flip. Hard timeout 600 s (10 minutes) regardless.

4.4 Documentation Session button (GPIO 23, Pin 16)

Toggles a "documentation session" — the container that bundles together photos, videos, and voice notes captured during a period of work.

Single press:

- If no session open → insert row in `sessions` table, status `open`, with timestamp and a label like *"Session 2026-05-09 14:32"*.
- If a session is open → update row to `status='closed'` with `ended_at = now()`.

800 ms cooldown absorbs hardware bounce — the second toggle within that window is silently ignored (otherwise the same finger-press could open + close in milliseconds).

4.5 Voice button (GPIO 22, Pin 15)

Press-to-toggle voice note. **Not** hold-to-record.

Press 1: Start recording. The handler subscribes to the AudioBridge mic stream and accumulates raw 16 kHz PCM into a buffer.

Press 2: Stop. The accumulated PCM is wrapped in a WAV header, uploaded to Supabase storage, and a row is inserted in `session_assets` (attached to the open session, or in `_orphan/` if none).

800 ms cooldown prevents bounce from auto-cancelling.

Mic-conflict guard: Voice notes refuse to start while an Agent or SOS AI session is active (they share the I²S mic — concurrent consumers would corrupt both streams).

4.6 Visual button (GPIO 25, Pin 22)

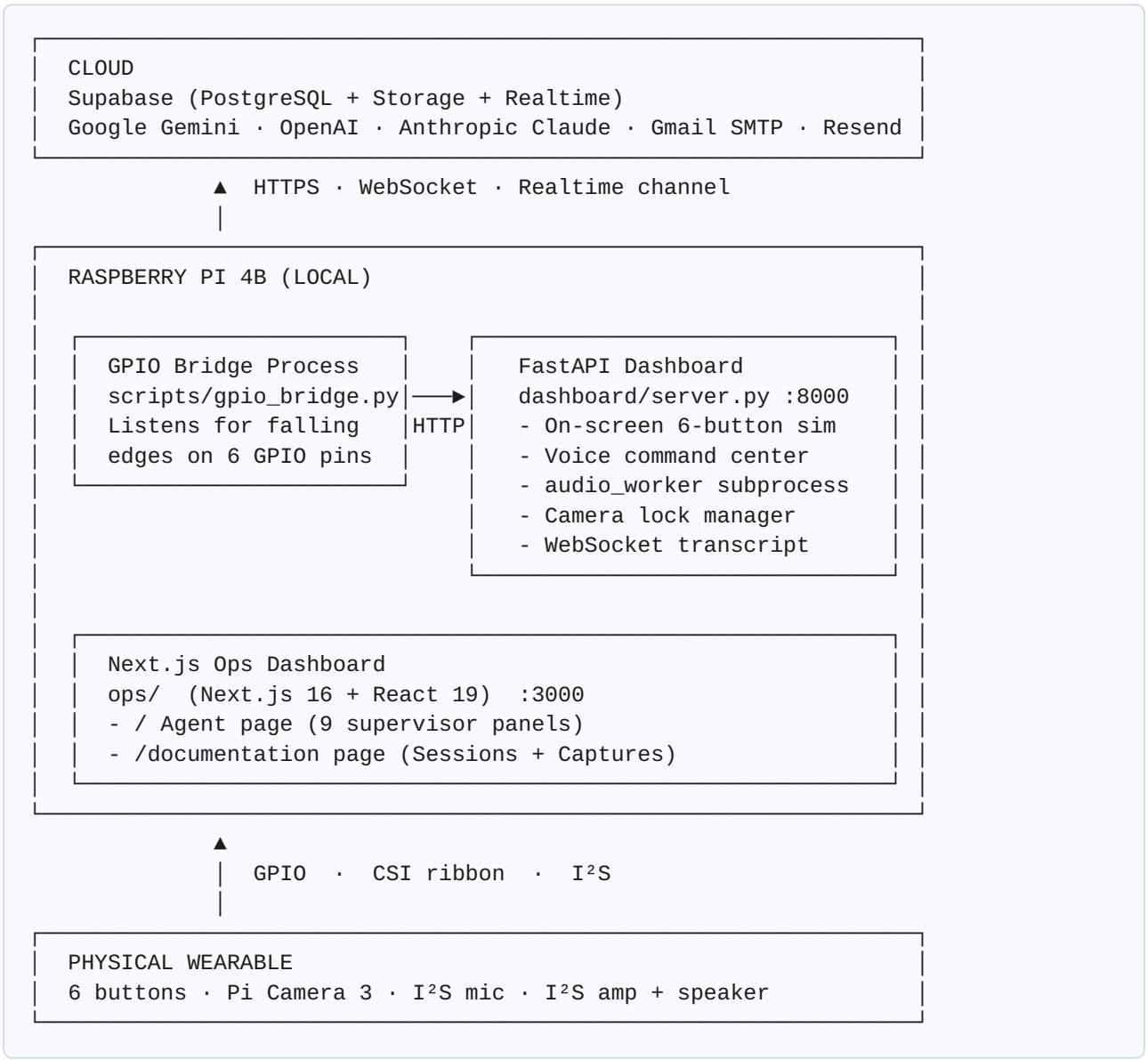
Hands-free camera capture.

Single press: Take a JPEG photo (1024×576) via picamera2. Upload to `session-assets/<session_id>/photo_<ts>.jpg`. Insert row in `session_assets` with `kind='photo'`.

Double press (within 500 ms): Record a 5-second MP4 via `rpicam-vid --codec libav --libav-format mp4`. Upload similarly with `kind='video'`. **No audio in videos for now** — the I²S mic is held exclusively by `audio_worker`, so a parallel `arecord` would fail. Audio mux through AudioBridge is a planned improvement.

---

5. The Software Stack



5.1 Languages

Language	Where it runs	Role
Python 3.13	Raspberry Pi	Backend FastAPI server, GPIO handling, AI integration, audio capture, picamera2 control, all cloud API calls
TypeScript	Browser (compiled to JS by Next.js)	Ops dashboard — 9 live data panels with full CRUD
JavaScript (vanilla)	Browser	Voice command center — the simulated wearable UI on the worker side
HTML / Tailwind CSS 4	Browser	Layout and styling for both dashboards
Bash	Raspberry Pi	System orchestration, launch scripts
SQL	Supabase	Database schema, indexes, realtime publication

5.2 Frameworks and Libraries

Layer	Technology	Why
HTTP / WebSocket server	**FastAPI** + **Uvicorn**	Python-native, async-first, perfect for real-time event handling
Frontend (supervisor)	**Next.js 16** + **React 19**	Modern fast dev experience
Styling	**Tailwind CSS 4**	Utility-first, no separate CSS files
DB client (Python)	**supabase-py**	Realtime subscriptions and storage uploads
DB client (browser)	**@supabase/supabase-js**	Live subscriptions and direct CRUD
GPIO handling	**RPi.GPIO 0.7.2** + **lgpio** + **gpiozero**	Falling-edge interrupts with 200 ms debounce
Camera	**picamera2** (still photos) + **rpicam-vid** (video)	Official modern Pi Camera library
Audio capture/ playback	**sounddevice** + **arecord/aplay** + custom AudioBridge	Direct I²S device access
Voice AI	**google-genai** (Gemini Live) + **OpenAI SDK** (Realtime)	Bidirectional streaming voice + vision
Email	**smtplib** (Gmail) + **resend** (Resend, alternative)	Default Gmail; Resend supported
Image utilities	**Pillow**, **opencv-python-headless**	Resizing, encoding, JPEG compression

6. The AI Brains

VisionLink uses three different cloud AI providers, each chosen for a specific strength.

6.1 Google Gemini Live (default)

- Provider: Google AI Studio
- Model: `gemini-2.5-flash` and `gemini-3.1-flash-live-preview` for streaming
- Used for: Agent and Agent+Vision sessions (default unless overridden)
- Why: Multimodal native (audio + image + text in one stream), fastest first-token latency, lowest cost per minute. Excellent at general conversation and tool calling.

6.2 OpenAI GPT Realtime 2

- Provider: OpenAI
- Model: `gpt-realtime-2`
- Used for: Optional alternative for Agent / Agent+Vision; **always used for SOS** regardless of other settings.
- Why: Better instruction-following for tightly scripted personas like *calm emergency operator*. Voice "Cedar", reasoning_effort "medium", speed 1.2× by default.

6.3 Anthropic Claude

- Provider: Anthropic
- Model: Claude Sonnet 4.6 (planned for email drafting)
- Used for: Research target — would route the SOS or Agent voice transcript to Claude, which writes a polished email body that Gmail SMTP delivers.
- Why: Best long-form writing quality of the three. Maintains a professional tone better.

6.4 Soniox (planned alternative STT)

- Provider: Soniox
- Used for: Fallback speech-to-text if Gemini Live's built-in transcription is insufficient. Stub today.

---

7. The Tools the AI Can Call

Both Gemini and OpenAI sessions expose the same six tools. Tools are how the AI takes real action — looking up data, writing rows, sending email.

#	Tool	What it does	DB
1	`lookup_component(query)`	Search the parts catalog by part code (exact) then name (fuzzy). Returns up to 3 rows with torque spec, maintenance interval, safety notes.	Read
2	`log_incident(description, category?, severity?, location?)`	Insert into `incidents` whenever the worker reports something wrong.	Write
3	`mark_task_complete(task_query)`	Fuzzy-match against open `worker_tasks` and mark complete.	Write
4	`get_my_assignments(include_complete?)`	List the worker's pending or all tasks.	Read
5	`request_part(part_query, quantity?, urgency?, reason?)`	Insert into `part_requests` for procurement.	Write
6	`send_report(report_name, recipient_role?, ..., custom_message?)`	Look up template, look up recipient, auto-inject live data, send via Gmail SMTP, log to `sent_reports`.	Read + Write + Email

Three rules in the system prompt prevent failure modes:

1. **Never lie about completing actions.** "I logged it" requires the tool to have actually been called.
2. **Verb triggers.** Words like *log*, *mark*, *send*, *request* trigger the matching tool immediately.
3. **No info-collection trap.** AI never asks more than one clarifying question — commits to action with reasonable defaults.

---

8. The Database

Supabase = managed PostgreSQL with file storage, realtime push, and SDKs for Python and JavaScript.

8.1 Tables

Table	Purpose
`components`	Parts catalog (read by `lookup_component`)
`worker_tasks`	Tasks assigned to a worker
`incidents`	Reported safety / equipment / leak / damage events
`part_requests`	Procurement queue
`managers`	Recipient list for emails (role → email)
`report_templates`	HTML email templates with placeholders
`sent_reports`	Audit log of every email sent
`sessions`	Documentation sessions (Session button)
`session_assets`	Photos, videos, voice notes (Visual / Voice buttons)
`sos_events`	SOS panic events with live transcript
`wearable_settings`	Singleton config row — providers, vision mode, SOS tunables, worker identity

8.2 Storage

Bucket `session-assets` holds every captured photo, video, voice note, and SOS frame. Public URLs go directly into the database — no signing needed for the demo.

8.3 Realtime

Every table is in the `supabase_realtime` publication. The ops dashboard subscribes once per table and gets `INSERT` / `UPDATE` / `DELETE` events over WebSocket within ~200 ms.

8.4 Row-Level Security

Disabled for the demo. Production deployment would re-enable RLS with auth-based per-row policies.

---

9. The Two Web Dashboards

9.1 Voice Command Center (worker simulator)

- URL: `http://visionlink.local:8000` (mDNS — works on any network the Pi joins)
- Tech: FastAPI + vanilla HTML/JS/Tailwind
- Audience: developer or worker testing the wearable
- Has: 6 on-screen buttons matching the physical buttons, live AI transcript, recent captures, debug overrides

9.2 Ops Dashboard (supervisor)

- URL: `http://visionlink.local:3000` (mDNS — works on any network the Pi joins)
- Tech: Next.js 16 + React 19 + Tailwind 4
- Audience: factory supervisor

Two pages, switchable from top nav:

`/` — **Agent page** with nine realtime panels: Incidents, Tasks, Parts, Components, Managers, Report Templates, Sent Reports, SOS, Wearable Settings.

`/documentation` — **Documentation page** showing what the worker captured: stats strip + Sessions panel (list of shifts with asset counts) + Captures panel (tabbed grid: All / Photos / Videos / Voice notes / Orphans, with click-to-zoom and inline playback).

---

10. Hosting and Cloud Services

10.1 What runs locally on the Pi

Process	Port	Purpose
`uvicorn dashboard.server:app`	8000	Backend + voice command center + websocket transcript
`next dev` (in `ops/`)	3000	Ops supervisor dashboard
`python3 -u scripts/gpio_bridge.py`	—	Translates physical button presses into HTTP calls
`audio_worker` (sub-process of FastAPI)	—	I²S mic + speaker, owns the AudioBridge

To start everything from scratch:

```
cd ~/Desktop/visionlink
nohup python3 -m uvicorn dashboard.server:app --host 0.0.0.0 --port 8000 > /tmp/vl_dashboard.log 2>&1 & disown

cd ~/Desktop/visionlink/ops
nohup npx next dev > /tmp/vl_ops.log 2>&1 & disown

cd ~/Desktop/visionlink
nohup python3 -u scripts/gpio_bridge.py > /tmp/vl_bridge.log 2>&1 & disown
```

10.2 What runs in the cloud

Service	Provider	Purpose	Auth
Supabase project	Supabase (managed)	DB, storage, realtime	`SUPABASE_URL` + `SUPABASE_SERVICE_KEY` + `SUPABASE_ANON_KEY`
Gemini Live API	Google	Voice + vision AI	`GEMINI_API_KEY`
OpenAI Realtime API	OpenAI	Voice + vision AI, SOS	`OPENAI_API_KEY`
Anthropic API (planned)	Anthropic	Email drafting	`ANTHROPIC_API_KEY`
Gmail SMTP	Google	Email delivery	`SMTP_EMAIL` + `SMTP_APP_PASSWORD`
Resend (alt)	Resend	Email delivery	`RESEND_API_KEY`
Soniox (alt)	Soniox	STT	`SONIOX_API_KEY`

All API keys live in `/.env` at the project root (gitignored). A template ships as `.env.example`.

---

11. Open Issues (action list)

These are known problems with the current build. None are blockers for the demo, but they should be fixed in priority order.

11.1 Visual button: single click triggers VIDEO instead of photo (HIGH)

Symptom: One press of the Visual button starts a 5-second video recording instead of capturing a photo.

Root cause: The button has mechanical bounce in the 100-300 ms range. Hardware debounce (`BUTTON_DEBOUNCE_MS = 200` in `config.py`) filters bounces ≤200 ms, but bounces 200-499 ms apart pass through and get classified as a "double press" within the 500 ms double-press window → fires the video handler.

Proposed fix: Add a per-button cooldown to `b2_take_photo` and `b2_record_video` in `src/subsystems/button_handlers.py`, modelled on the session button's existing 800 ms cooldown. First gesture wins; bounce-induced doubles within 800 ms are silently dropped.

```
# In _ButtonState dataclass:
last_b2_action_at: float = 0.0

# At top of both b2 handlers:
B2_COOLDOWN_S = 0.8
now = time.time()
if now - _state.last_b2_action_at < B2_COOLDOWN_S:
    return {"action": "ignored", "reason": "cooldown"}
_state.last_b2_action_at = now
```

11.2 Camera lock race when a video is requested twice in flight (MEDIUM)

Symptom: Second `rpicam-vid` subprocess errors with `'imx708': Unable to set controls: Device or resource busy`. First video saves successfully; second errors out.

Proposed fix: Add `video_recording: bool` flag to `_ButtonState`. At top of `b2_record_video`, refuse if true. Set true before subprocess; reset in `finally`. (The flag is already added to `_ButtonState` per a recent edit; the guard logic still needs to be wired in.)

11.3 Visual button videos have no audio (DEFERRED)

Symptom: Recorded MP4s play with silent video.

Root cause: `audio_worker` holds the I²S mic exclusively. A parallel `arecord` would fail with EBUSY, so videos use `rpicam-vid` alone.

Proposed fix: Route audio capture through the existing AudioBridge during video recording, then mux with `ffmpeg`. Significant work — defer unless explicitly required.

11.4 Voice command center simulator: voice button still uses hold (LOW)

Symptom: On-screen voice button uses hold-to-record gestures but the physical voice button is press-to-toggle. Inconsistent during browser testing.

Proposed fix: Change `data-gestures="hold"` to `data-gestures="single"` on the voice button cell in `dashboard/static/index.html`.

11.5 "voice note too short" message is misleading (LOW)

Symptom: When the mic is dead (e.g. `~/.asoundrc` corrupt) every voice press shows *"voice note too short — discarded"*. Confusing.

Proposed fix: In `b3_voice_note_end`, if `len(buf) == 0`, log *"voice note received no audio — check mic"* instead.

---

12. Recurring Hardware Quirks (good to know)

12.1 `~/.asoundrc` disappears intermittently

The ALSA config file at `~/.asoundrc` has been observed to vanish mid-session, breaking I²S audio. The dashboard now detects this on audio-worker spawn and restores a symlink. If voice notes start coming back as silence/EBUSY, check `~/.asoundrc` first.

12.2 AudioBridge mic queue must be drained per consumer

The bridge has a 200-second-deep mic queue. Voice notes and AI sessions need to call `drain_mic()` on start, otherwise stale audio from the previous consumer plays back into the new one.

12.3 Tactile button bounce varies by individual switch

Different physical switches in the same batch bounce differently. Some need 50 ms debounce, others need 200 ms+. The current global `BUTTON_DEBOUNCE_MS = 200` is a compromise; per-button software cooldowns (Issue 11.1) handle the rest.

---

13. Common Project Review Questions

Q. What database does VisionLink use?

PostgreSQL, hosted via Supabase (managed). The `supabase_realtime` extension pushes row changes over WebSocket to the dashboards.

Q. How does the wearable talk to the AI?

A bidirectional WebSocket from the Pi to either Google Gemini Live or OpenAI Realtime. Audio streams up; the AI's voice and tool calls stream down. Tool calls are intercepted on the Pi, executed against Supabase or Gmail, and the result returns through the same WebSocket.

Q. Why two AI providers?

Different strengths. Gemini is cheaper, faster, and multimodal-native. OpenAI Realtime is better at strict-instruction personas like the SOS operator. The supervisor switches the default for Agent / Agent+Vision from the Wearable Settings panel without restarting anything.

Q. What language is the backend?

Python 3.13 with FastAPI for HTTP/WebSocket. AsyncIO for concurrent tasks.

Q. What language is the supervisor dashboard?

TypeScript on Next.js 16 with React 19 and Tailwind 4. Live updates via Supabase Realtime over WebSocket.

Q. How do photos and videos get to the supervisor?

Pi uploads directly to Supabase Storage (S3-compatible). Row in `session_assets` points to the storage path. Supervisor's browser subscribes to the table, gets the new row, fetches the public URL, and displays inline.

Q. What happens if the network is down?

Buttons still register locally, but Supabase inserts fail and the worker hears an error tone. AI sessions can't connect. Captures don't currently buffer for retry across power cycles — that's an acknowledged improvement target.

Q. What's the GPIO bridge?

A 100-line Python sidecar (`scripts/gpio_bridge.py`) that listens for falling edges on the six button pins via RPi.GPIO and translates each gesture (single, double) into the same HTTP call the on-screen simulator makes. Runs as a separate process so software changes in one can't break the other.

Q. How are double-clicks detected?

Two falling edges within 500 ms (`DOUBLE_PRESS_WINDOW`) on the same pin, both passing the 200 ms hardware debounce. First edge starts a timer; if a second arrives, fire double. If timer expires, fire single.

Q. What's the SOS panic mode different from a normal AI session?

Five things: always uses OpenAI · different system prompt · inserts a DB row at trigger · auto-snaps frames every 10 s · has remote shutoff (supervisor flips `resolved=true`, wearable polls every 2 s and tears down).

Q. How much does this cost to run?

Per ~10 minutes of voice + vision: Gemini ~€0.15, OpenAI ~€0.40. Supabase + storage + email free tier. A full SOS event ~€0.30.

Q. Where is the source code?

`https://github.com/alaamjaish/visionlink` on the `openai-sdk` branch.

---

14. Glossary

Term	What it means
BCM	Broadcom GPIO numbering. Used everywhere in this doc.
GPIO	General-Purpose Input/Output. Pi's programmable pins.
Falling edge	A signal transition from high (3.3 V) to low (0 V). What the Pi listens for.
Pull-up resistor	Internal resistor holding an input pin at 3.3 V when nothing's connected. Pi has these on every GPIO.
I²S	Inter-IC Sound — a digital audio bus. Three wires per direction.
CSI	Camera Serial Interface — the ribbon cable connector on the Pi for the Pi Camera.
WebSocket	Persistent two-way connection between browser and server. Used for live transcripts and AI streaming.
Realtime	Supabase's WebSocket-based push of database changes.
Tool call	When the AI invokes a function (like `lookup_component`) that runs server-side and returns a result.
VAD	Voice Activity Detection — model's heuristic for end of speech.
TTS	Text-to-Speech.
STT	Speech-to-Text.
RLS	Row-Level Security — PostgreSQL feature for per-row permissions. Disabled in this demo.
systemd	Linux service manager. Auto-start unit lives at `visionlink.service`.
Supabase storage bucket	S3-compatible cloud file store. VisionLink uses one bucket: `session-assets`.

End of master reference.